Data Structures (in C++)

- Binary Search Trees -

부산대학교 정보·의생명공학대학
정보컴퓨터공학부

Binary Search Trees

Binary Search Trees

- **Search Tree**
  - A search tree is a tree data structure that can be used to implement ordered maps and dictionaries

A data type for storing key-value pairs that are sorted according to some total order

- **Binary Search Tree**
  - A binary search tree is a binary tree T such that each internal node v of T stores an entry (k, x) such that:
    - Keys stored at nodes in the left subtree of v are less than or equal to k
    - Keys stored at nodes in the right subtree of v are greater than or equal to k.
  - Binary search trees are nonempty proper binary trees

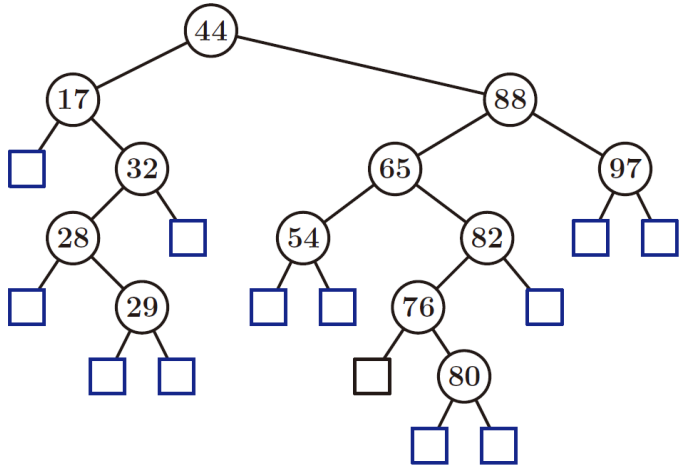


Figure 10.1: A binary search tree T representing a map with integer keys.

Binary Search Trees: Searching

- **Searching**
  - Views the tree T as a decision tree
  - Checks at each internal node v of T whether the search key k is less than, equal to, or greater than the key of the node v
  - Returns a node position w as follows:

    - w is an internal node and w's entry has key equal to k
    - w is an external node representing k's proper place in an inorder traversal of $T(v)$, but k is not a key contained in $T(v)$

Binary Search Trees: Searching

- **Searching**

Algorithm TreeSearch(k, v):
  if T.isExternal(v) **then**
    return v
  if $k <$ key(v) **then**
    return TreeSearch$(k, T$.left$(v))$
  else if $k >$ key(v) **then**
    return TreeSearch$(k, T$.right$(v))$
  return v $\{$we know $k =$ key$(v)\}$

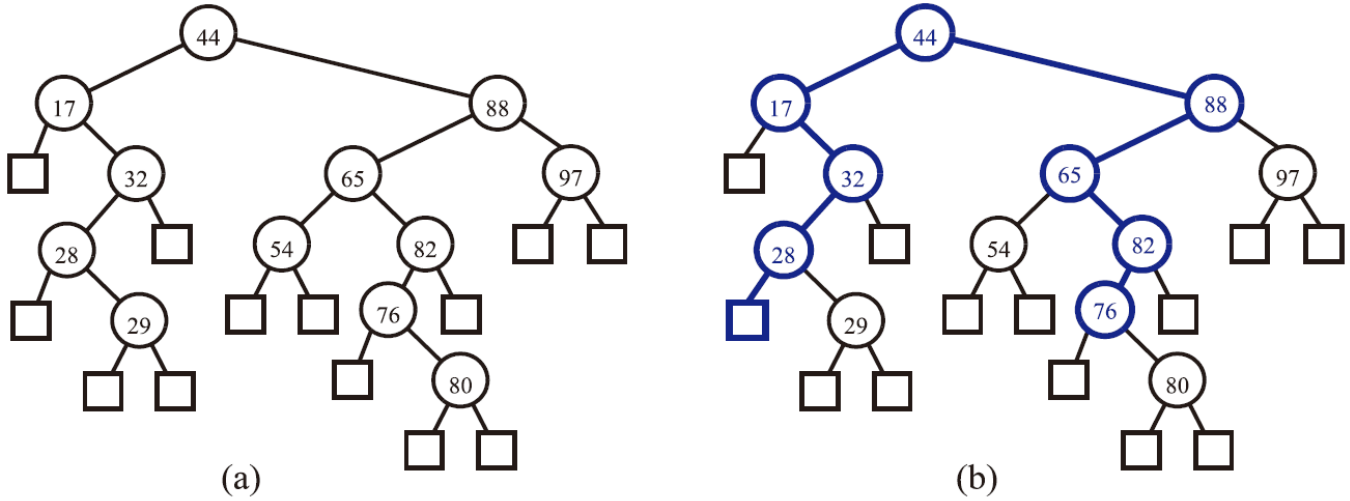


Figure 10.2: (a) A binary search tree T representing a map with integer keys; (b) nodes of T visited when executing operations find(76) (successful) and find(25) (unsuccessful) on M. For simplicity, we show only the keys of the entries.

Binary Search Trees: Searching

- **Analysis of Binary Tree Searching**
  - *TreeSearch()* performs a constant number of primitive operations for each recursive call
  - Called on the nodes of a path of T that starts at the root and goes down one level each time
  - Searching runs in $O(h)$ where h is the height of T (*cf.*, The number of nodes on a path is bounded by $(h+1)$)
  - *findAll(k)* takes $O(h+s)$ where s is the number of entries with key k

Algorithm TreeSearch(k, v):
  if T.isExternal(v) **then**
    return v
  if $k <$ key(v) **then**
    return TreeSearch$(k, T$.left$(v))$
  else if $k >$ key(v) **then**
    return TreeSearch$(k, T$.right$(v))$
  return v $\{$we know $k =$ key$(v)\}$

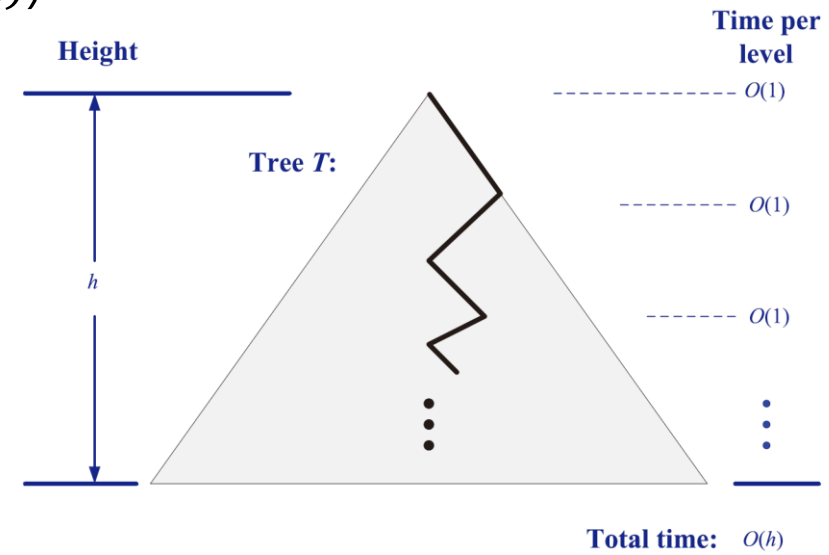


Figure 10.3: The running time of searching in a binary search tree. We use a standard visualization shortcut of viewing a binary search tree as a big triangle and a path from the root as a zig-zag line.

Binary Search Trees: Update

- **Insertion**
  - *TreeInsert()* traces a path from *T*'s root to an external node

insertAtExternal(v,e): Insert the element e at the external node v, and expand v to be internal, having new (empty) external node children; an error occurs if v is an internal node.

Algorithm TreeInsert(k,x,v):
Input: A search key k, an associated value, x, and a node v of T
Output: A new node w in the subtree $T(v)$ that stores the entry (k,x)

```
w ← TreeSearch(k, v)
if T.isInternal(w) then
    return TreeInsert(k, x, T.left(w)) {going to the right would be correct too}
T.insertAtExternal(w, (k,x))          {this is an appropriate place to put (k,x)}
return w
```

Code Fragment 10.2: Recursive algorithm for insertion in a binary search tree.

Binary Search Trees: Update

- **Insertion**
  - *TreeInsert()* traces a path from *T*'s root to an external node

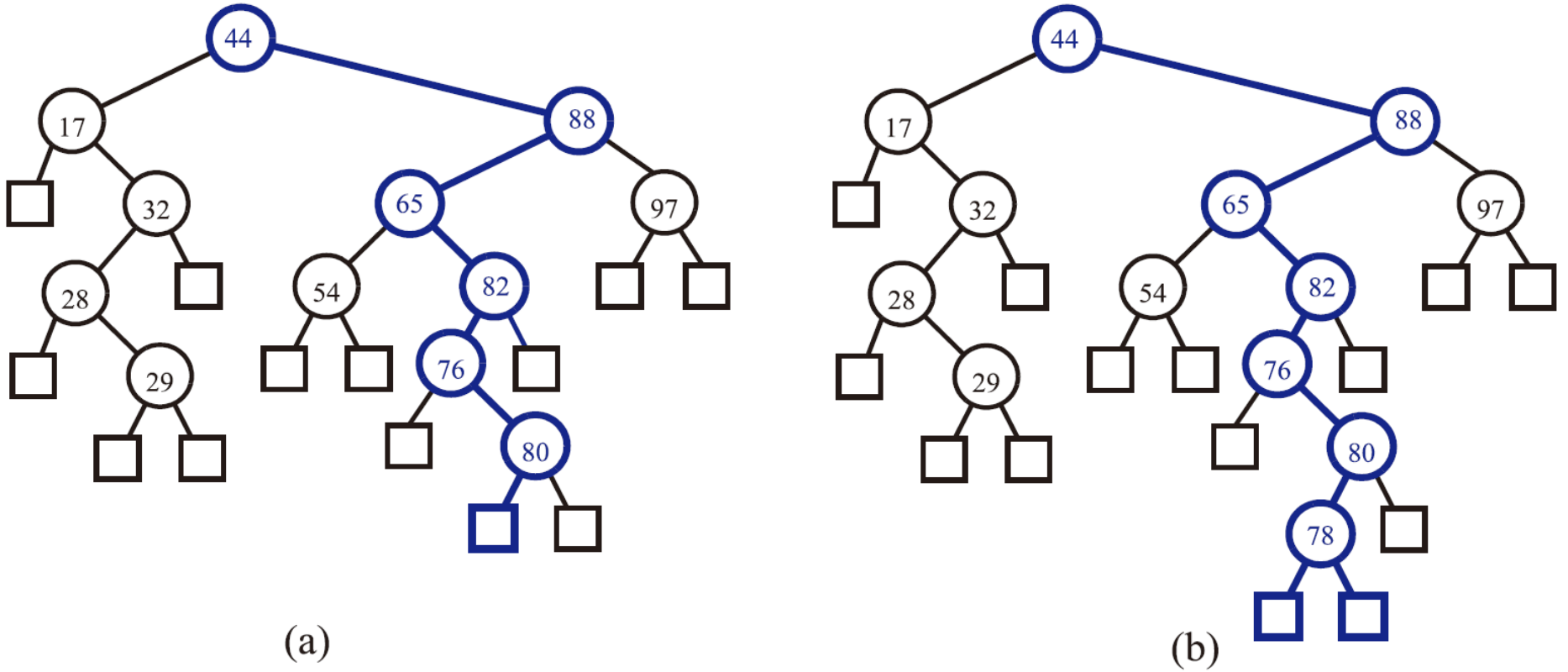


Figure 10.4: Insertion of an entry with key 78 into the search tree of Figure 10.1: (a) finding the position to insert; (b) the resulting tree.

Binary Search Trees: Update

- **Removal**
  - Should not create any holes in the tree T

To keep the tree proper

removeAboveExternal(v): Remove an external node v and its parent, replacing v's parent with v's sibling; an error occurs if v is not external.

- If one of the children of node w is an external node, say node z, we simply remove w and z from T by means of operation removeAboveExternal(z) on T. This operation restructures T by replacing w with the sibling of z, removing both w and z from T. (See Figure 10.5.)
- If both children of node w are internal nodes, we cannot simply remove the node w from T, since this would create a "hole" in T. Instead, we proceed as follows (see Figure 10.6):
  - We find the first internal node y that follows w in an inorder traversal of T. Node y is the left-most internal node in the right subtree of w, and is found by going first to the right child of w and then down T from there, following the left children. Also, the left child x of y is the external node that immediately follows node w in the inorder traversal of T.
  - We move the entry of y into w. This action has the effect of removing the former entry stored at w.
  - We remove nodes x and y from T by calling removeAboveExternal(x) on T. This action replaces y with x's sibling, and removes both x and y from T.

Search for the "smallest" entry from the right subtree

Binary Search Trees: Update

- **Removal**
  - Should not create any holes in the tree T

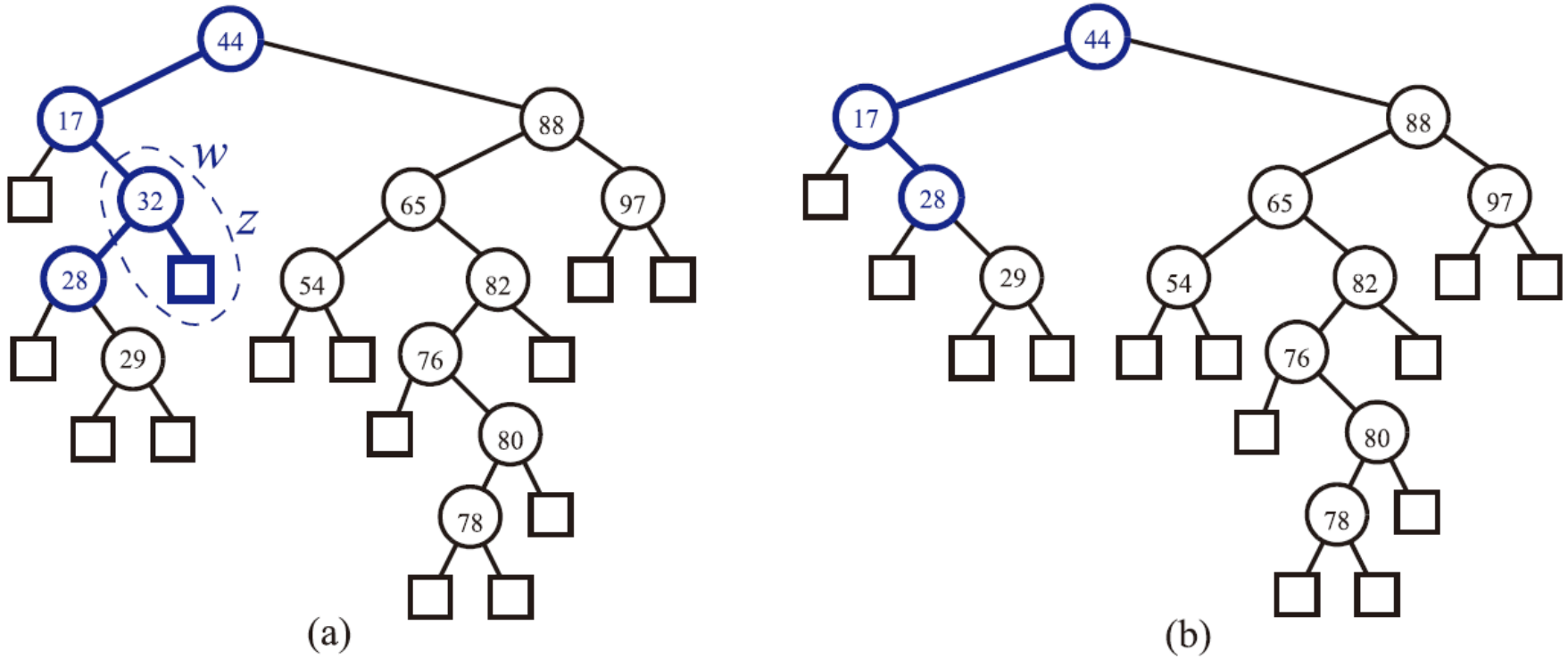


Figure 10.5: Removal from the binary search tree of Figure 10.4b, where the entry to remove (with key 32) is stored at a node (w) with an external child: (a) before the removal; (b) after the removal.

Binary Search Trees: Update

- **Removal**
  - Should not create any holes in the tree T

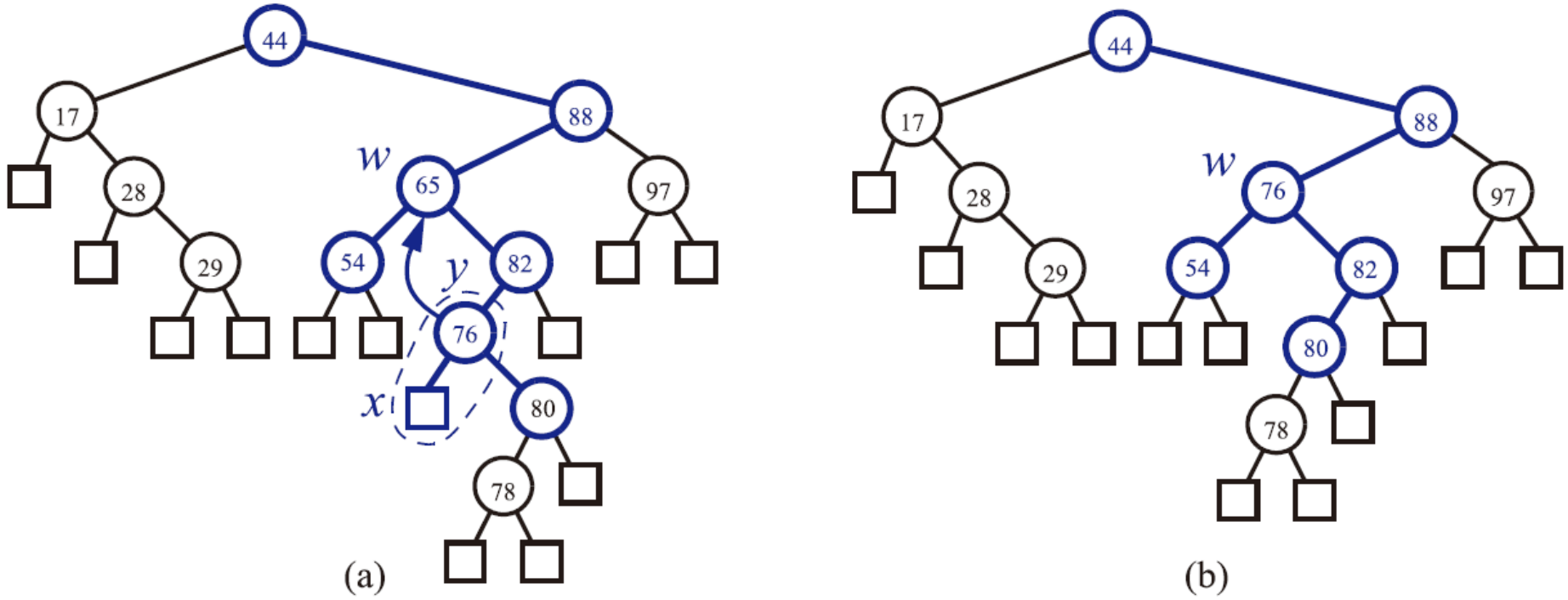


Figure 10.6: Removal from the binary search tree of Figure 10.4b, where the entry to remove (with key 65) is stored at a node (w) whose children are both internal: (a) before the removal; (b) after the removal.

Performance of Binary Search Trees

- $O(1)$ time at each node
- The number of nodes visited is proportional to the height h of T
  - The best case: $h = \lceil \log(n+1) \rceil$
  - The worst case: $h = n$
- The *expected* height of a binary search tree is $O(\log n)$

Operation	*Time*
size, empty	$O(1)$
find, insert, erase	$O(h)$

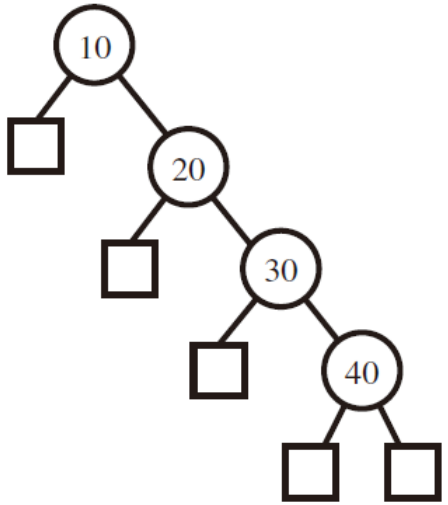
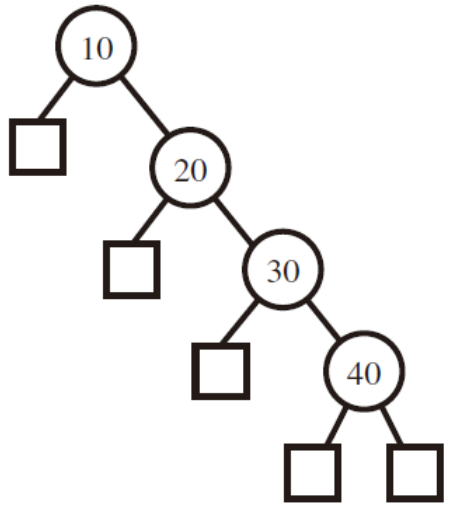


Figure 10.7: Example of a binary search tree with linear height, obtained by inserting entries with keys in increasing order.

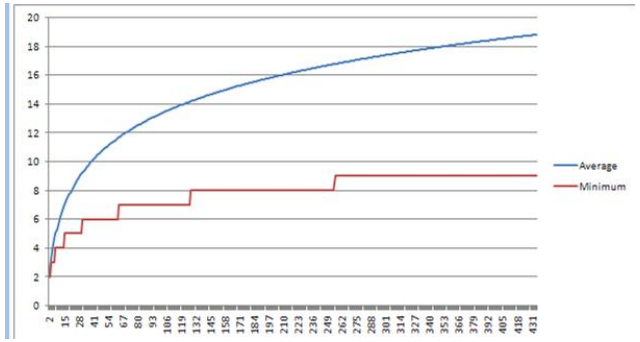
Performance of Binary Search Trees

- Example: The *expected* height of a binary search tree

1st	2nd	3rd	4th	Height	1st	2nd	3rd	4th	Height
10	20	30	40	**4**	30	10	20	40	**3**
10	20	40	30	**4**	30	10	40	20	**3**
10	30	20	40	**3**	30	20	10	40	**3**
10	30	40	20	**3**	30	20	40	10	**3**
10	40	20	30	**4**	30	40	10	20	**3**
10	40	30	20	**4**	30	40	20	10	**3**
20	10	30	40	**3**	40	10	20	30	**4**
20	10	40	30	**3**	40	10	30	20	**4**
20	30	10	40	**3**	40	20	10	30	**3**
20	30	40	10	**3**	40	20	30	10	**3**
20	40	10	30	**3**	40	30	10	20	**4**
20	40	30	10	**3**	40	30	20	10	**4**



- 80 / 24 = 3.3333



https://stackoverflow.com/questions/861393/average-height-of-a-binary-search-tree

Binary Search Trees

- **C++ Implementation**

```
template <typename K, typename V>
class Entry {                                   // a (key, value) pair
public:
  typedef K Key;                                // public types
  typedef V Value;                              // key type
public:                                         // value type
  Entry(const K& k = K(), const V& v = V())     // public functions
    : _key(k), _value(v) { }                    // constructor
  const K& key() const { return _key; }         // get key (read only)
  const V& value() const { return _value; }     // get value (read only)
  void setKey(const K& k) { _key = k; }         // set key
  void setValue(const V& v) { _value = v; }     // set value
private:                                        // private data
  K _key;                                       // key
  V _value;                                     // value
};
```

```
template <typename E>
class SearchTree {                                          // a binary search tree
public:                                                     // public types
  typedef typename E::Key K;                                // a key
  typedef typename E::Value V;                              // a value
  class Iterator;                                           // an iterator/position
public:                                                     // public functions
  SearchTree();                                             // constructor
  int size() const;                                         // number of entries
  bool empty() const;                                       // is the tree empty?
  Iterator find(const K& k);                                // find entry with key k
  Iterator insert(const K& k, const V& x);                  // insert (k,x)
  void erase(const K& k) throw(NonexistentElement);         // remove key k entry
  void erase(const Iterator& p);                            // remove entry at p
  Iterator begin();                                         // iterator to first entry
  Iterator end();                                           // iterator to end entry
protected:                                                  // local utilities
  typedef BinaryTree<E> BinaryTree;                         // linked binary tree
  typedef typename BinaryTree::Position TPos;               // position in the tree
  TPos root() const;                                        // get virtual root
  TPos finder(const K& k, const TPos& v);                   // find utility
  TPos inserter(const K& k, const V& x);                    // insert utility
  TPos eraser(TPos& v);                                     // erase utility
  TPos restructure(const TPos& v)                           // restructure
      throw(BoundaryViolation);
private:                                                    // member data
  BinaryTree T;                                             // the binary tree
  int n;                                                    // number of entries
public:
  // ...insert Iterator class declaration here
};
```

Binary Search Trees

- **C++ Implementation (Binary Tree)**

```
struct Node {                                   // a node of the tree
  Elem    elt;                                  // element value
  Node*   par;                                  // parent
  Node*   left;                                 // left child
  Node*   right;                                // right child
  Node() : elt(), par(NULL), left(NULL), right(NULL) { } // constructor
};
```

```
class Position {                                // position in the tree
private:
  Node* v;                                      // pointer to the node
public:
  Position(Node* _v = NULL) : v(_v) { }         // constructor
  Elem& operator*()                             // get element
    { return v->elt; }
  Position left() const                         // get left child
    { return Position(v->left); }
  Position right() const                        // get right child
    { return Position(v->right); }
  Position parent() const                       // get parent
    { return Position(v->par); }
  bool isRoot() const                           // root of the tree?
    { return v->par == NULL; }
  bool isExternal() const                       // an external node?
    { return v->left == NULL && v->right == NULL; }
  friend class LinkedBinaryTree;                // give tree access
};
typedef std::list<Position> PositionList;       // list of positions
```

Code Fragment 7.18: Class Position implementing a position in a binary tree. It is nested in the public section of class LinkedBinaryTree.

Binary Search Trees

- **C++ Implementation (Binary Tree)**

```
template <typename E>                    // base element type
class BinaryTree<E> {                    // binary tree
public:                                  // public types
  class Position;                        // a node position
  class PositionList;                    // a list of positions
public:                                  // member functions
  int size() const;                      // number of nodes
  bool empty() const;                    // is tree empty?
  Position root() const;                 // get the root
  PositionList positions() const;        // list of nodes
};
```

Code Fragment 7.16: An informal interface for the binary tree ADT (not a complete C++ class).

Binary Search Trees

- **C++ Implementation**

```
class Iterator {                                  // an iterator/position
private:
  TPos v;                                         // which entry
public:
  Iterator(const TPos& vv) : v(vv) { }            // constructor
  const E& operator*() const { return *v; } // get entry (read only)
  E& operator*() { return *v; }                   // get entry (read/write)
  bool operator==(const Iterator& p) const        // are iterators equal?
    { return v == p.v; }
  Iterator& operator++();                         // inorder successor
  friend class SearchTree;                        // give search tree access
};
```

```
/* SearchTree⟨E⟩ :: */                                  // inorder successor
Iterator& Iterator::operator++() {
  TPos w = v.right();
  if (w.isInternal()) {                               // have right subtree?
    do { v = w; w = w.left(); }                       // move down left chain
    while (w.isInternal());
  }
  else {
    w = v.parent();                                   // get parent
    while (v == w.right())                            // move up right chain
      { v = w; w = w.parent(); }
    v = w;                                            // and first link to left
  }
  return *this;
}
```

Advances the iterator from a given position to its inorder successor

Binary Search Trees

- **C++ Implementation**
  - The rightmost node should return *end()*
  - Introduce a special sentinel called the *super root*
  - The root of the binary search tree called the *virtual root* is the left child of the *super root*
  - *end()* is equal to the *super root*

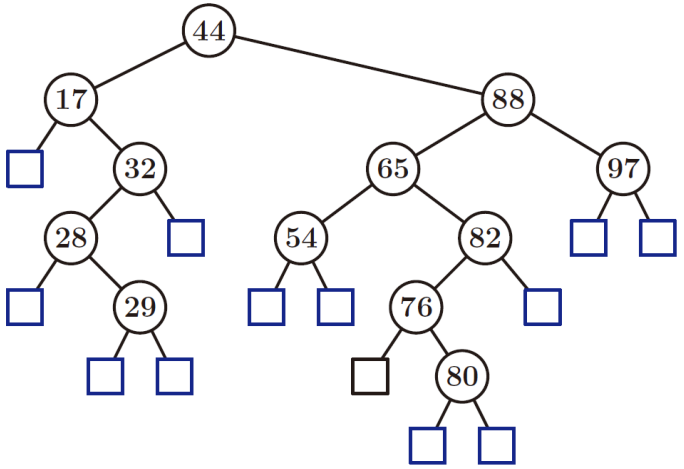


Figure 10.1: A binary search tree T representing a map with integer keys.

```
/* SearchTree⟨E⟩ :: */                          // inorder successor
 Iterator& Iterator::operator++() {
  TPos w = v.right();
  if (w.isInternal()) {                         // have right subtree?
    do { v = w; w = w.left(); }                 // move down left chain
    while (w.isInternal());
  }
  else {
    w = v.parent();                             // get parent
    while (v == w.right())                      // move up right chain
      { v = w; w = w.parent(); }
    v = w;                                      // and first link to left
  }
  return *this;
}
```

Binary Search Trees

- **C++ Implementation**
  - Constructor creates the *super root*

```
/* SearchTree⟨E⟩ :: */                          // constructor
  SearchTree() : T(), n(0)
    { T.addRoot(); T.expandExternal(T.root()); }  // create the super root

/* SearchTree⟨E⟩ :: */                          // get virtual root
  TPos root() const
    { return T.root().left(); }                 // left child of super root
```

Code Fragment 10.7: The constructor and the utility function root. The constructor creates the super root, and root returns the virtual root of the binary search tree.

```
/* SearchTree⟨E⟩ :: */                          // iterator to first entry
  Iterator begin() {
    TPos v = root();                            // start at virtual root
    while (v.isInternal()) v = v.left();        // find leftmost node
    return Iterator(v.parent());
  }

/* SearchTree⟨E⟩ :: */                          // iterator to end entry
  Iterator end()
    { return Iterator(T.root()); }              // return the super root
```

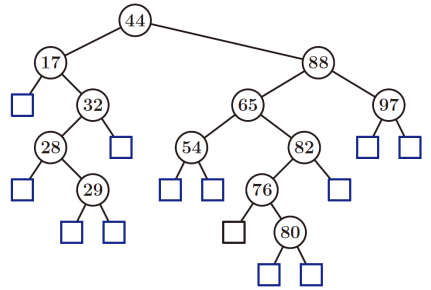


Figure 10.1: A binary search tree T representing a map with integer keys.

Binary Search Trees

- **C++ Implementation**

```
/* SearchTree⟨E⟩ :: */                                    // find utility
  TPos finder(const K& k, const TPos& v) {
    if (v.isExternal()) return v;                          // key not found
    if (k < v->key()) return finder(k, v.left());          // search left subtree
    else if (v->key() < k) return finder(k, v.right());    // search right subtree
    else return v;                                         // found it here
  }

/* SearchTree⟨E⟩ :: */                                    // find entry with key k
  Iterator find(const K& k) {
    TPos v = finder(k, root());                            // search from virtual root
    if (v.isInternal()) return Iterator(v);                // found it
    else return end();                                     // didn't find it
  }
```

Code Fragment 10.9: The functions of SearchTree related to finding keys.

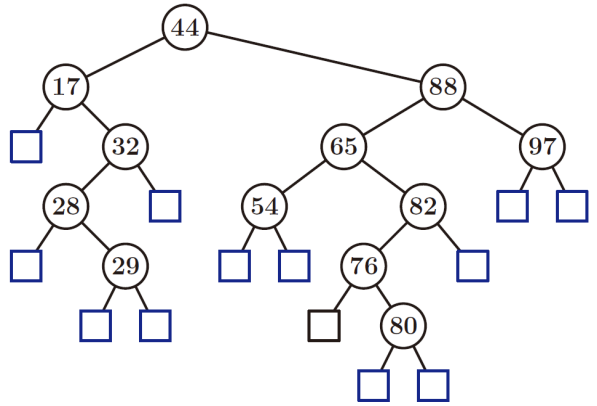


Figure 10.1: A binary search tree T representing a map with integer keys.

Binary Search Trees

- **C++ Implementation**

```
/* SearchTree⟨E⟩ :: */                          // insert utility
  TPos inserter(const K& k, const V& x) {
    TPos v = finder(k, root());                 // search from virtual root
    while (v.isInternal())                      // key already exists?
      v = finder(k, v.right());                 // look further
    T.expandExternal(v);                        // add new internal node
    v->setKey(k); v->setValue(x);               // set entry
    n++;                                        // one more entry
    return v;                                   // return insert position
  }

/* SearchTree⟨E⟩ :: */                          // insert (k,x)
  Iterator insert(const K& k, const V& x)
    { TPos v = inserter(k, x); return Iterator(v); }
```

Code Fragment 10.10: The functions of SearchTree for inserting entries.

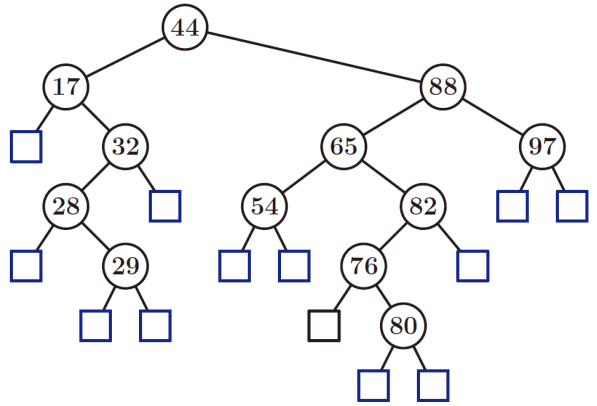


Figure 10.1: A binary search tree T representing a map with integer keys.

Binary Search Trees

- **C++ Implementation**

```
/* SearchTree⟨E⟩ :: */                                  // remove utility
  TPos eraser(TPos& v) {
    TPos w;
    if (v.left().isExternal()) w = v.left();            // remove from left
    else if (v.right().isExternal()) w = v.right();     // remove from right
    else {                                              // both internal?
      w = v.right();                                    // go to right subtree
      do { w = w.left(); } while (w.isInternal());      // get leftmost node
      TPos u = w.parent();
      v->setKey(u->key()); v->setValue(u->value()); // copy w's parent to v
    }
    n--;                                                // one less entry
    return T.removeAboveExternal(w);                    // remove w and parent
  }

/* SearchTree⟨E⟩ :: */                                  // remove key k entry
  void erase(const K& k) throw(NonexistentElement) {
    TPos v = finder(k, root());                         // search from virtual root
    if (v.isExternal())                                 // not found?
      throw NonexistentElement("Erase of nonexistent");
    eraser(v);                                          // remove it
  }

/* SearchTree⟨E⟩ :: */                                  // erase entry at p
  void erase(const Iterator& p)
    { eraser(p.v); }
```

Code Fragment 10.11: The functions of SearchTree involved with removing entries.

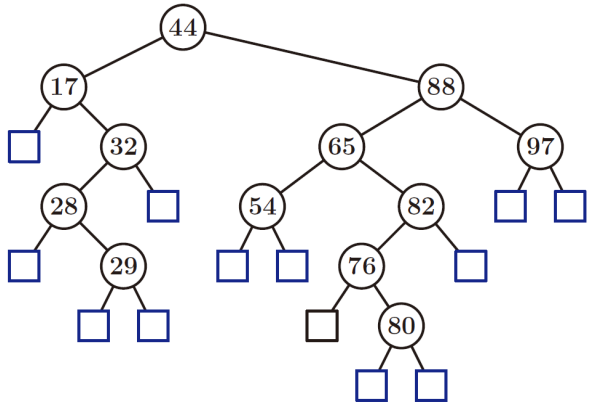


Figure 10.1: A binary search tree T representing a map with integer keys.